Activity: Group REST API Enhancement and Multi-Client Implementation

- Cuario, John Lloyd M
- Loterte, Eljohn Paulo C.
- Napay, Victor Noel A.
- Rebancos, Sean Jerve Ll.
- Relos, Kiel Hedrix V.

Submission Requirementsx
Source code (server + client)

API documentation (1–2 pages)

Individual journals (per member)

Part 3: Group Demonstration

Each group must present:

1. API overview (endpoints, parameters, logic)

ENDPOINTS & PARAMETERS

```
## 1. User Authentication (Login)
Validates a user's credentials against the database. If successful, returns basic non-sensitive profile information to initialize the client application.

- **Method:** `POST`
- **Endpoint:** `/api.php`
- **Headers:** `Content-Type: application/json`
- **Request Body Configuration:**
```json
{
  "username": "your_username",
  "password": "your_password"
}
```

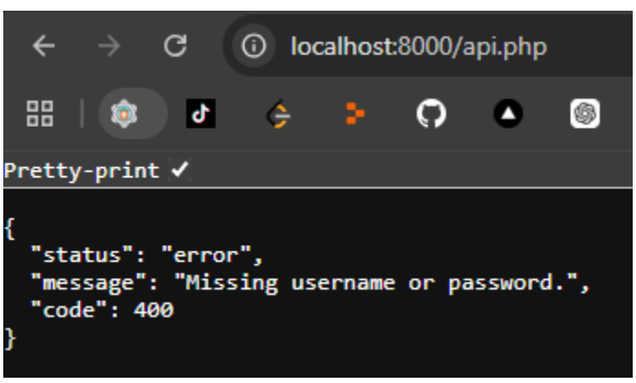
Success Responses

HTTP 200 OK - Successful Login
Returned when the username and password match.
```json
{
  "status": "success",
  "message": "Login successful!",
  "userProfile": {
    "name": "User's Full Name",
    "role": "admin"
  },
  "code": 200
}
```
```

SYSTEM LOGIC

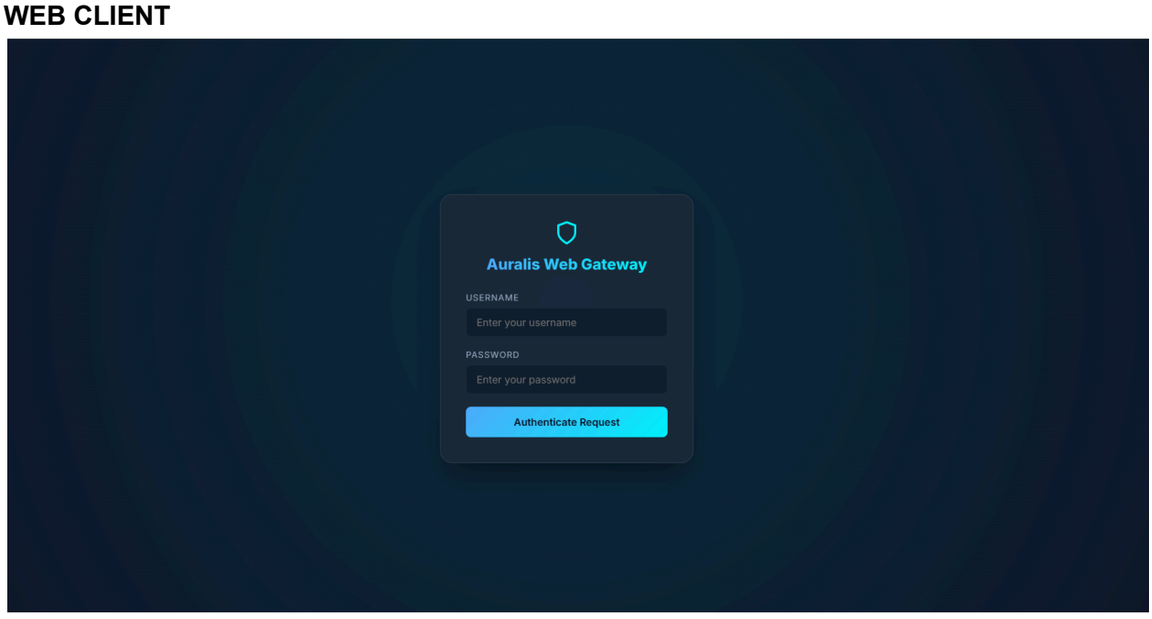
```php
// If the database is offline or XAMPP isn't running, we kill the script immediately
// and return a 500 Internal Server Error so the frontend knows what broke.
if ($conn->connect_error) {
    echo json_encode(["status" => "error", "message" => "Database connection failed. Did you start MySQL in XAMPP?", "code" =>
    http_response_code(500);
    exit;
}

// We artificially pause the script for 1 second to simulate real-world internet latency.
// This allows our frontends to successfully render their "Loading" animations for the presentation.
sleep(1);

// --- 4. VALIDATION PIPELINE ---
// Check if the user accidentally pressed login while leaving fields entirely blank.
if (!$username || !$password) {
    echo json_encode(["status" => "error", "message" => "Missing username or password.", "code" => 400]);
    http_response_code(400); // 400 Bad Request
    exit;
}

// --- 5. SECURE DATABASE QUERYING ---
// Instead of injecting the username directly into the SQL (which causes SQL Injection hacks),
// we use "Prepared Statements". We prepare the template, then bind the 's' (string) safely.
$stmt = $conn->prepare("SELECT password, name, role, status FROM users WHERE username = ?");
$stmt->bind_param("s", $username);
$stmt->execute();
$result = $stmt->get_result();

// If the database returns 0 rows, it means the username does not exist.
if ($result->num_rows === 0) {
    echo json_encode(["status" => "error", "message" => "User account not found.", "code" => 404]);
    http_response_code(404); // 404 Not Found
    $stmt->close();
    $conn->close();
    exit;
}
```

THE NETWORK OUTPUT

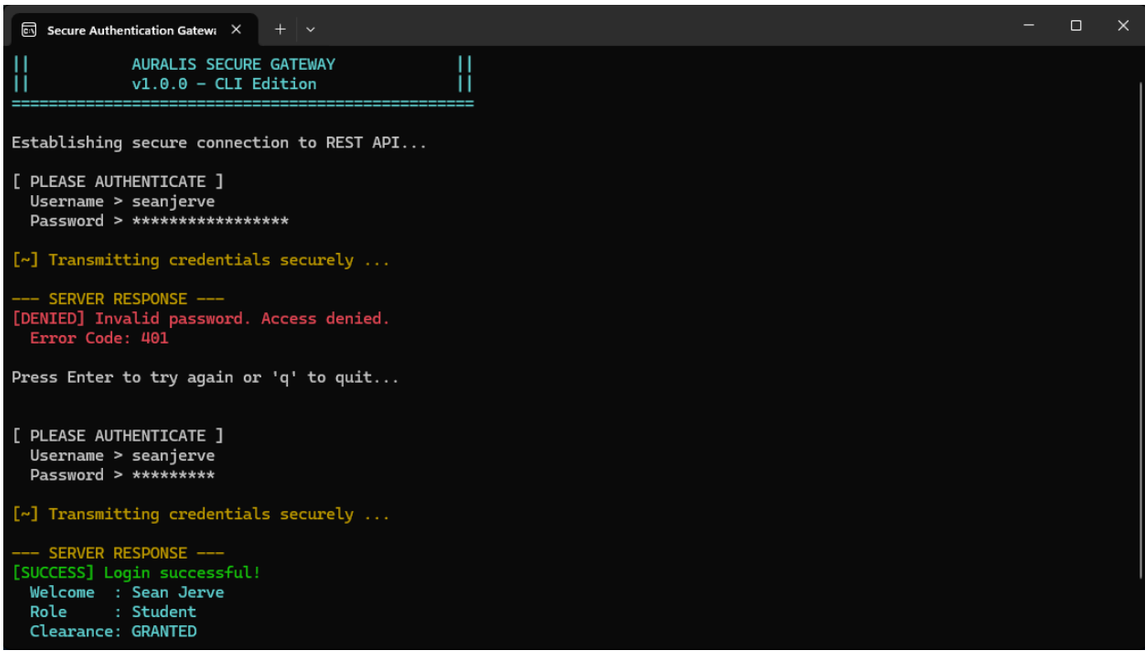


2. Client application

WEB CLIENT

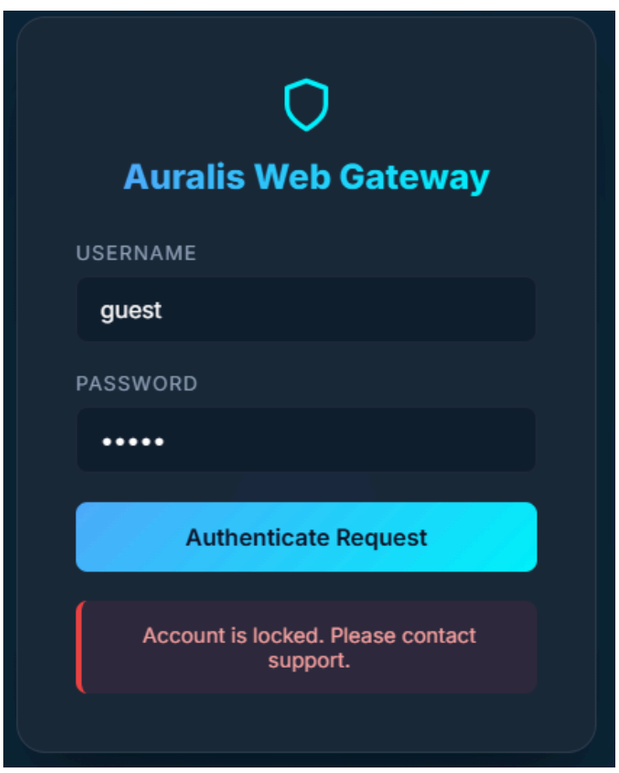


C# Command Line (CLI)

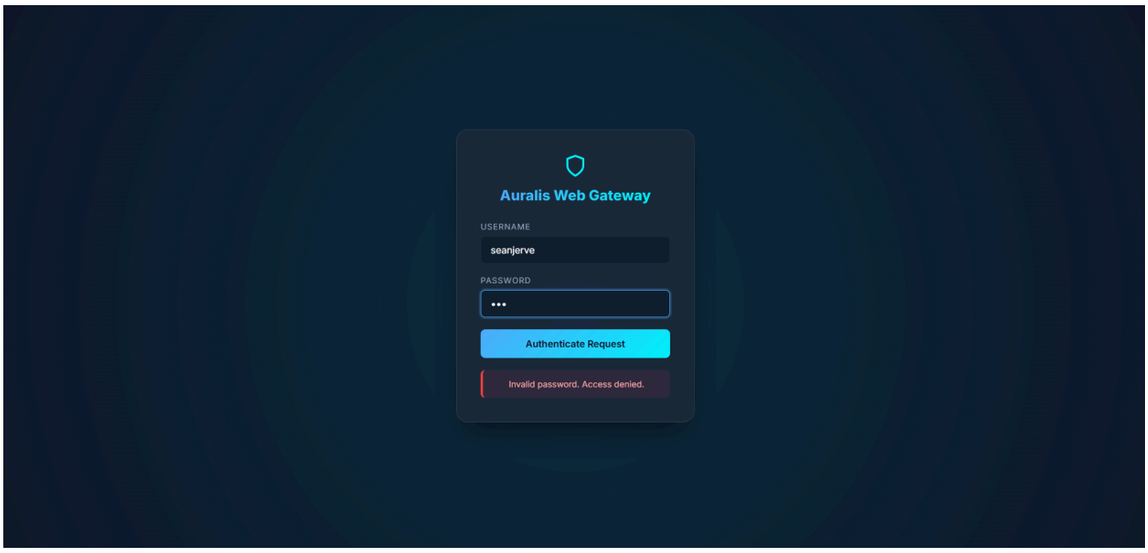


3. Live demo (request → response)

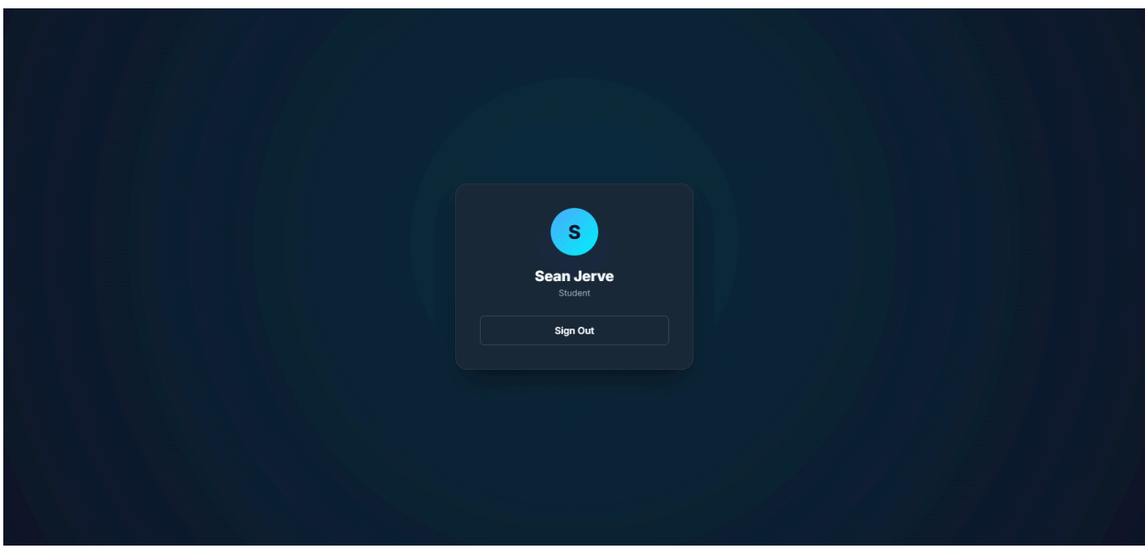
LOGIN 1.0



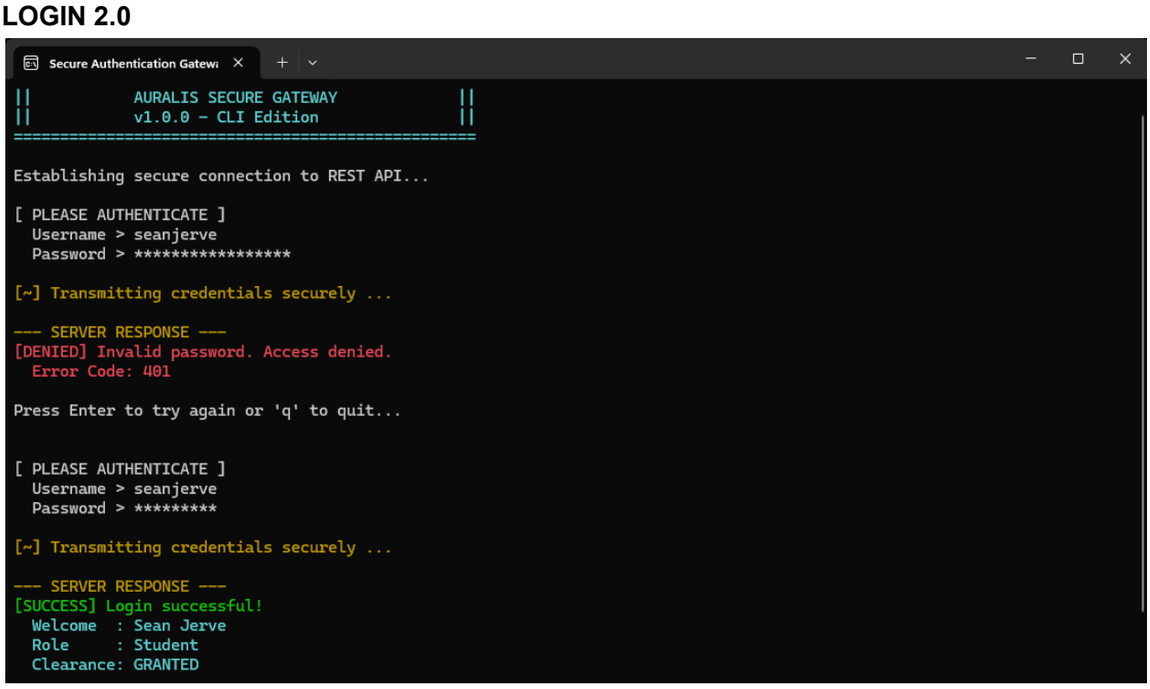
LOGIN 1.1



LOGIN 1.2



LOGIN 2.0



Part 4: Individual Journal (Required)

Name: Eljohn Paulo Loterte
Role: Lead Backend Developer (API Server & Database)

Specific Tasks Performed
The specific task I performed in this project was architecting and developing the server-side logic in our `api.php` file. I was specifically tasked with creating a secure, RESTful API that both our C# application and our web application could safely verify users against. I configured the MySQL Database connection parameters and built the logic pipeline that securely authenticates JSON login requests into verifiable database actions.

Code Contributions
I wrote the entirety of the `api.php` logic. In which I first established the universal REST standards by utilizing `header("Content-Type: application/json")` so all clients knew how to read the data. Then constructed the JSON parser using `file_get_contents('php://input')` which allowed the script to cleanly extract incoming passwords and usernames from `POST` requests. Furthermore, I implemented robust backend security utilizing `mysqli` Prepared Statements (`$stmt->bind_param("s", $username);`) protecting our `auralis_db` from SQL Injection vulnerabilities. Finally, I authored the logic matrix returning distinct error payloads and exact HTTP status codes ranging from `400 Bad Request` to `200 Success` depending on the database logic verification.

Challenges Encountered
A challenge during backend implementation I encountered was the "Cross-Origin Resource Sharing" (CORS) restriction. In which whenever our web client attempted to POST to the API, the browser blocked the request for security reasons. Additionally, I also struggled with how the PHP engine stopped or crashed completely if incomplete JSON data was submitted by the client (for example, missing a password field entirely).

How they Contributed to Solving Problems
I solved the server crashing issue by strictly adding validation conditionals (`if (!$username || !$password)`) directly after decoding the JSON payload, which ejected a `"status" => "error"` packet rather than triggering a fatal PHP crash. I actively resolved the CORS issue by carefully documenting and inserting the explicit `Access-Control-Allow-Origin: *` strings combined with handling native browser `OPTIONS` preflight checks, creating a perfectly stabilized environment allowing all web fetches to be securely authorized without browser blockages.

Name: Sean Jerve Rebancos
Role:C# Desktop Client Developer

Specific Tasks Performed

- My unique responsibility within our group was engineering the desktop application client using C# (`Program.cs`). My primary objective was to build a secure, command-line interface (CLI) application capable of securely networking with the PHP REST API to execute the remote authentication sequences seamlessly without needing an organic web browser.

Code Contributions

I structured and meticulously developed the core networking loop utilizing the C# `System.Net.Http.HttpClient` library. I authored the `AuthenticateUser` method specifically converting user console string inputs into structured Dictionary packets using internal `JavaScriptSerializer.Serialize()` methodologies, successfully mocking standard HTTP JSON POST request bodies. When the server successfully responded, my code deserialized the raw network strings (`ReadAsStringAsync()`) back into C# variables. I also engineered the primary CLI user interface—managing `Console.ForegroundColor` states, and specifically implementing a custom `ReadPassword()` method that actively masked users' keyboard strokes with asterisks (`*`) for security.

Challenges Encountered

Developing network code within a synchronous Console environment rapidly produced architectural challenges. If the server lagged during a request, the entire console application would freeze permanently. Additionally, handling complex JSON arrays in C# logic required dynamically casting untyped string outputs to readable dictionaries, which often threw `NullReferenceExceptions` when the server arbitrarily returned distinct error objects lacking "userProfile" fields.

How they Contributed to Solving Problems

I bypassed the synchronous freezing completely by refactoring the `Main` thread into an asynchronous `MainAsync().Wait()` task pool, implementing `async/await` methodology for the `client.PostAsync()` functions to ensure the network operations processed sequentially without choking the CPU. I actively prevented JSON parsing exceptions by utilizing `dict.ContainsKey()` validity checks prior to executing the UI rendering, seamlessly handling the diverse error states—whether the user was met with an "Account Locked" or "Wrong Password" string depending entirely on the API's distinct return configurations.

Name: John Lloyd M. Cuario

Role: Web Client Developer (Frontend Logic)

Specific Tasks Performed

-My main responsibility to the group was developing the frontend Javascript logic to connect our HTML interface with the backend PHP server. I handled the core client-side data flow in app.js: capturing user inputs, formatting them into JSON payloads, transmitting them securely via API requests, and dynamically updating the DOM (Document Object Model) based on the server's responses without refreshing the browser.

Code Contributions

-I built the core network interactions using the native Fetch API. Using the DOMContentLoaded event hook, I attached listeners to our forms and utilized e.preventDefault() to prevent standard page reloads, giving the interface a smoother, app-like feel. For authentication, I serialized the username and password data with JSON.stringify() and routed POST requests to our local PHP server (localhost:8000). Once the server responded, I parsed the payload with await response.json() and mapped the retrieved data directly to the UI, dynamically populating elements like profileName.textContent and the avatarLetter.

Challenges Encountered

-The main challenge was timing. My code was trying to display the user profile before the server had actually finished sending the data, which caused "undefined" errors to show up on the screen. Additionally, switching the screen from the "Login" view to the "Profile" view caused visual glitches because I was trying to manually delete and recreate HTML elements on the fly.

How I Contributed to Solving Problems

-To fix the timing issue, I used async/await to force the Javascript to pause and wait for the server's data to fully arrive before updating the screen. To fix the glitchy screen transitions, I stopped trying to delete raw HTML. Instead, I used CSS to simply hide the login screen (.classList.add('hidden')) and reveal the dashboard. This made the transition from login to profile instant and perfectly smooth.

Name:Kiel Hedrix Relos
Role: UI/UX Designer

Specific Tasks Performed
My responsibility to this task is to design the integration of both client applications across fundamentally varied frameworks (HTML/CSS vs C# Console Arrays). Our team developed multiple different platforms accessing the same centralized database API, ensuring brand consistency, designing seamless user interfaces, and generating seamless visual feedback interactions that were completely assigned to my control.

Code Contributions
Inside the Javascript environment, I authored the entirety of `style.css` defining the Client's modern identity. Instead of pulling external framework templates, I manually programmed custom variable palettes (`:root`), Flexbox viewport centering mechanisms, and smooth glass-like UI border radiuses defining true modern authentication layouts. For the C# environment, I significantly overhauled the Command Line string presentation matrices inside `Program.cs` designing the `[~] Transmitting credentials securely ...` animation and rendering precise `Console.ForegroundColor` toggles (Green for validations, Red for API timeouts, Cyan for headers) allowing an archaic CLI application to emulate premium security software aesthetics.

Challenges Encountered
Developing true responsive behaviors inside basic HTML required tedious scaling, specifically keeping the dynamically populated error cards (`.error`) consistently centered vertically regardless of window resolutions. Simulating organic UI animations without triggering CPU performance spikes in C# CLI also required intricate timing modifications since consoles fundamentally lack native animation rendering engines.

How they Contributed to Solving Problems
I rectified the C# constraints by strategically programming customized `Thread.Sleep(300)` iterating loops during the authentication payloads, producing a visually immersive "Loading..." text animation entirely masked under raw network wait-times. In the web CSS module, I incorporated robust CSS flex box integrations manipulating standard error messages (`#responseMessage`) into animated hidden blocks featuring custom margin transitions that slid cleanly into view *without* disrupting the broader form alignments validating both premium aesthetics and structural network reliability identically.

Name: Victor Noel Napay
Role: QA Tester & System Documentation

Specific Tasks Performed:
My agenda or role in this project was to precisely assure edge-case stress testing and central system documentation protocol. I tested whether our clients legitimately parsed the backend logic parameters flawlessly under unstable scenarios while simultaneously ensuring the academic instructions guiding code architectures were definitively structured and followed.

Code Contributions
I functioned heavily across standardizing our codebase structures by deploying comprehensive "Human-Readable" variable summaries and documentation block annotations physically across `app.js`, `Program.cs`, and `api.php`. In addition, I constructed the definitive `api_documentation.md`, directly translating the complex PHP HTTP `200/400/403` routing code into an easily digestible syntax format. My quality control metrics discovered and validated code modifications directly in the frontend checking algorithms verifying empty inputs were appropriately blocked on the CLI prior to triggering wasteful server POST packets.

Challenges Encountered
One of the challenges that I've encountered was in isolating complex bugs within multi-threaded systems which naturally proved incredibly problematic. When the C# command console unexpectedly executed error messages containing unformatted JSON brackets rather than raw strings, discerning whether the C# `JavaScriptSerializer` natively failed or whether the PHP API injected a fatal syntax parameter in its HTTP header return sequence was extremely difficult without independent environmental controls.

How they Contributed to Solving Problems
I established an isolated Postman protocol structure, effectively completely decoupling the frontend from the testing parameters. By independently dispatching structured JSON variants directly manipulating `username` parameters externally, I accurately proved our API server operated perfectly under stress. I discovered that the bugs specifically lied inherently inside the C# Dictionary mapping logic whenever the backend returned non-uniform schemas, permitting my engineering teammates to surgically inject `dict.ContainsKey()` fail safes correctly patching all client vulnerabilities entirely, enabling a flawless presentation.